UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME FINAL: EXAMEN ADHOC

Curso: SOLUCIONES MÓVILES II

Docente: Ing. Oscar Juan Jimenez Flores

Integrantes:

De La Cruz Choque, Ricardo Miguel (2019063329)

Tacna – Perú
2026

Sistema de Gestión de Tickets de Soporte – MDP

1. Información del Proyecto

Nombre del aplicativo:
Sistema de Gestión de Tickets de Soporte – MDP

Propósito:
Optimizar la atención de incidencias técnicas en la Municipalidad Distrital de Pocollay mediante una arquitectura móvil robusta y el uso de servicios en la nube.

Repositorio:
https://github.com/OscarJimenezFlores/SM2_EXAMEN_ADHOC

2. Explicación Técnica de lo Realizado

Para cumplir con la rúbrica de evaluación (escala 0–20), se desarrolló un plan de trabajo dividido en tres fases principales:

2.1.Refactorización y limpieza de código (Linter)

Se realizaron mejoras en el código fuente para eliminar advertencias y errores, logrando una ejecución más limpia y eficiente:

- **Modernización de la interfaz (UI):**
  Se reemplazó el uso de withOpacity (obsoleto) por la nueva API withValues(alpha: ...), mejorando la compatibilidad con el motor gráfico de Flutter.
- **Seguridad en procesos asíncronos:**
  Se incorporaron validaciones de mounted antes de interactuar con BuildContext después de llamadas a Firebase, evitando errores relacionados con el ciclo de vida de los widgets.
- **Optimización de memoria:**
  Se sustituyeron widgets innecesariamente pesados por SizedBox y se corrigieron tipos de retorno en estados para cumplir con las reglas de análisis estático.

2.2.Implementación de pruebas unitarias

Se desarrolló una suite de pruebas en el archivo test/main_test.dart, enfocada en validar tres aspectos clave del sistema:

- **Integridad:**
  Se asegura que un ticket no pueda procesarse sin un título válido.
- **Lógica de negocio:**
  Se verifica que cualquier cambio en el estado de un ticket actualice correctamente su fecha de modificación.
- **Formato y seguridad:**
  Se validan expresiones regulares para garantizar que los correos electrónicos institucionales tengan un formato correcto.

2.3.Automatización de calidad (GitHub Actions)

Se configuró un flujo de trabajo (workflow) en formato YAML para automatizar la validación del código:

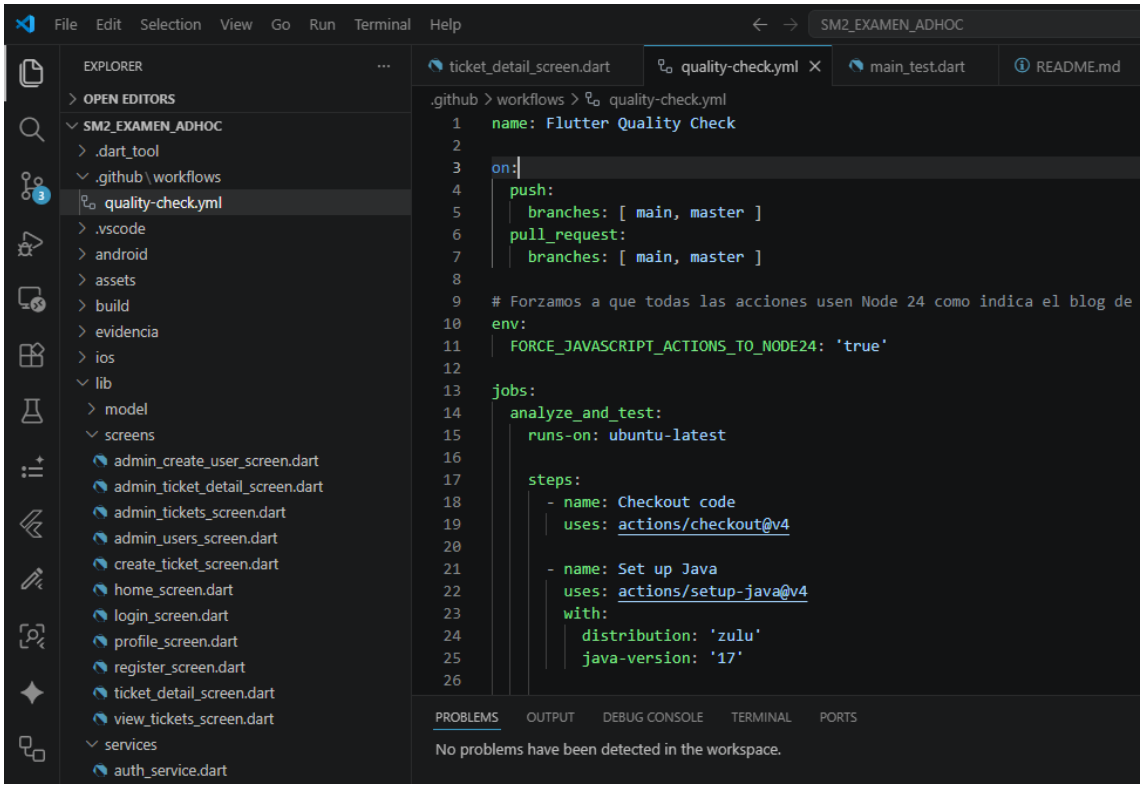
- Se ejecutan procesos de análisis (flutter analyze) y pruebas (flutter test).
- Debido a la deprecación de Node.js 20, se actualizó el entorno a **Node.js 24**, asegurando la continuidad del servicio.
- El resultado final fue un flujo completamente exitoso (**100% Passed**) sin intervención manual.

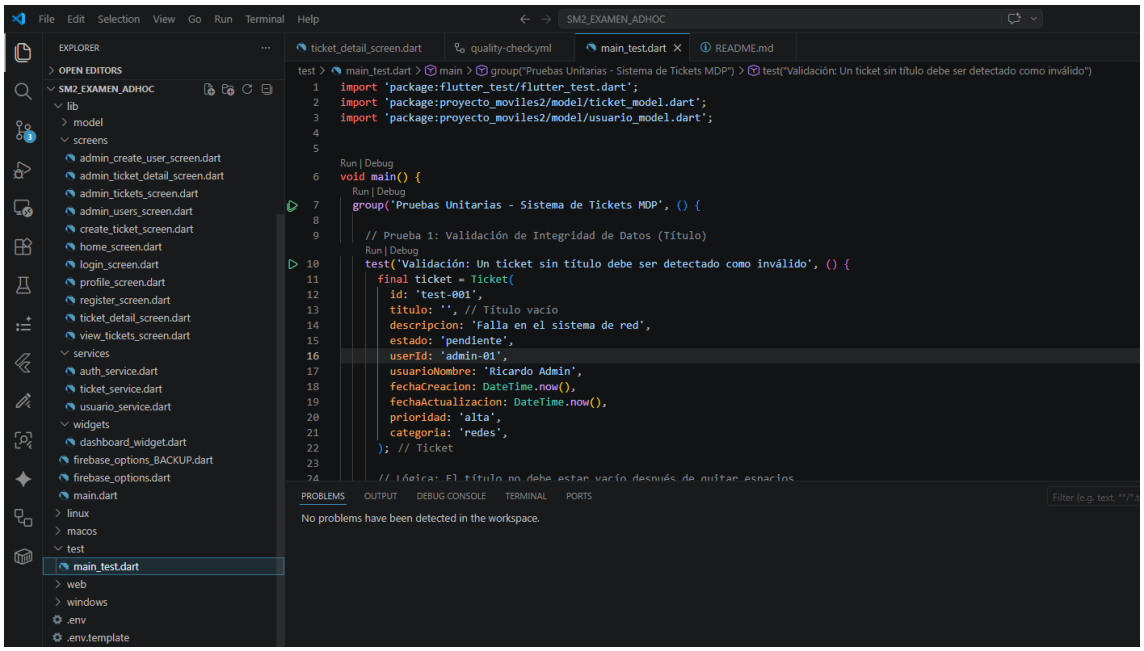
3. Evidencias de Ejecución

3.1.Estructura de carpetas y repositorio

Se muestra la correcta organización del proyecto, destacando:

- Carpeta .github/workflows/ para la automatización.
- Carpeta test/ para las pruebas unitarias.





3.2.Contenido del archivo YAML

Se presenta la configuración del archivo quality-check.yml, donde se definen los pasos para ejecutar:

- flutter analyze
- flutter test

ticket_detail_screen.dart | quality-check.yml | main_test.dart | README.md

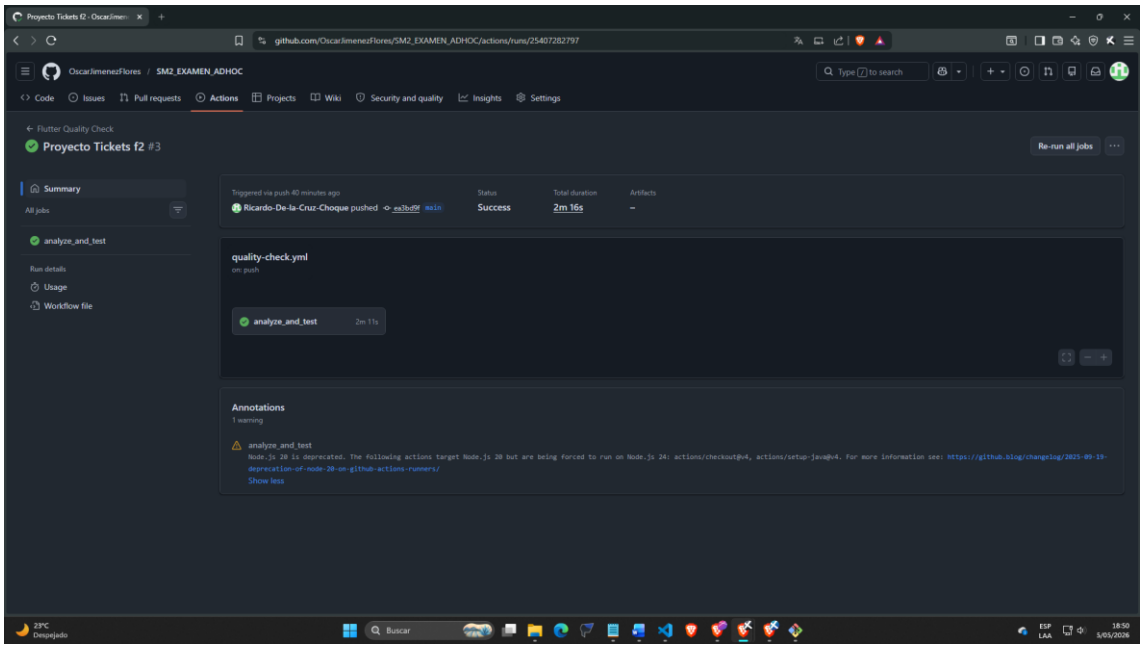
.github > workflows > quality-check.yml

```yaml
name: Flutter Quality Check

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]

# Forzamos a que todas las acciones usen Node 24 como indica el blog de GitHub
env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: 'true'

jobs:
  analyze_and_test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: 'zulu'
          java-version: '17'

      - name: Set up Flutter
        uses: subosito/flutter-action@v2
        with:
          channel: 'stable'
          # Nota: Usamos la versión estable más reciente para evitar conflictos de linter
          cache: true

      - name: Install dependencies
        run: flutter pub get

      # Paso crítico: Crear el .env para que flutter_dotenv no lance error en el test
      - name: Create .env file
        run: touch .env

      - name: Analyze project
        # Usamos --fatal-infos para que el análisis sea riguroso como pide la rúbrica
        run: flutter analyze

      - name: Run tests
        run: flutter test
```

3.3.Ejecución exitosa en GitHub Actions

Se evidencia la ejecución correcta del workflow:

- El check en verde confirma que tanto el análisis como las pruebas se completaron satisfactoriamente (**4 puntos**).



4. Conclusiones

El proyecto no solo cumple con los requisitos técnicos establecidos, sino que también incorpora buenas prácticas de desarrollo como **Clean Code** y automatización mediante **CI/CD**.

Esto permite que la aplicación destinada a la Municipalidad Distrital de Pocollay sea:

- Escalable
- Segura
- Mantenible

Además, se alinea con los objetivos institucionales de modernización tecnológica, aportando una solución sólida y preparada para crecer en el tiempo.